

Lab 3: Physical Coding Sublayer (PCS) 64b/66b Encoding & Scrambling

Hardware-in-the-Loop Simulation, Self-Synchronizing LFSR & 66-Bit Waveform Analysis

Zuñiga, Ivan Andrés

September 1, 2026

Fundación Fulgor

1. System Architecture & 64b/66b PCS Datapath

Hardware-in-the-Loop Setup:

- **Host Interface:** tap0 (10.0.0.1/24, MAC 02:00:00:00:00:01).
- **Target Interface:** tap1 in namespace ns_b (10.0.0.2/24, MAC 02:00:00:00:00:02).
- **RTL Stack (top.sv):** MAC TX/RX, 64b/66b PCS encoder/decoder, and 58-bit parallel LFSR scrambler.
- **C++ Driver (wrapper.cpp):** Non-blocking TAP driver with synchronous clocking across dual Linux namespaces.
- **Specification:** IEEE 802.3 Clause 82 (40G/100GBASE-R).

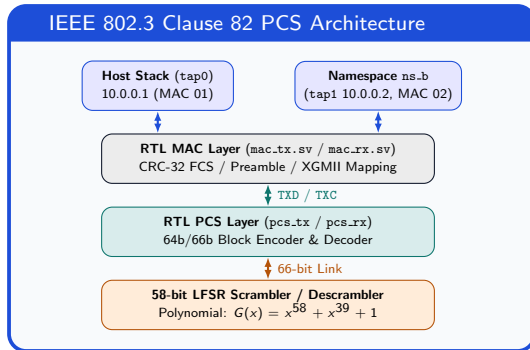


Figure 1: Hardware pipeline from Linux TAP devices to the 66-bit physical link.

2. Environment Setup & Traffic Generation

Execution Sequence:

1. Configure TAP Interfaces:

```
sudo ./setup_tap.sh
```

Configures tap0 and namespace ns_b with tap1.

2. Build and Run Verilator Emulator:

```
verilator --cc --trace --exe --build
```

```
wrapper.cpp top.sv
```

```
sudo ./obj_dir/emulator
```

```
--enable-scrambler
```

3. Capture Traffic:

```
sudo tcpdump -i tap0 -w
```

```
lab3_capture.pcap
```

4. Send Patterned Ping:

```
ping -c 2 -p cafe0003 -I tap0 10.0.0.2
```

```
sudo ./obj_dir/emulator --enable-scrambler
[NETNS] Cleaning up previous setup...
[NETNS] Creating namespace ns_b & TAP nodes...
[NETNS] Configuring IP addresses, MACs & link states...
[NETNS] Disabling hardware offloading & IPv6 (Clean Trace) ...
-----
[NETNS] Setup Complete:
  * Host (Host NetNS): tap0 → 10.0.0.1/24 (MAC 02:00:00:00:00:01, IPv6 OFF)
  * Node B (ns_b NetNS): tap1 → 10.0.0.2/24 (MAC 02:00:00:00:00:02, IPv6 OFF)
-----
[TAP] Bound successfully to interface: tap0
[TAP] Bound successfully to interface: tap1
-----
[EMULATOR] Dual TAP Node A (tap0) ↔ Node B (ns_b/tap1) Active
[EMULATOR] Scrambler Status: ENABLED (Polynomial G(x) = x58 + x49 + 1)
[EMULATOR] Waveform VCD trace logging enabled (waveform.vcd)
[EMULATOR] Press Ctrl+C to stop simulation cleanly.
=====
^C
[EMULATOR] Shutting down cleanly and closing waveform trace...

) sudo tcpdump -i tap0 -w lab3_capture.pcap
tcpdump: listening on tap0, link-type EN10MB (Ethernet), snapshot length 262144 bytes
^C4 packets captured
4 packets received by filter
0 packets dropped by kernel

) ping -c 2 -p cafe0003 -I tap0 10.0.0.2
PATTERN: 0xcafe0003
PING 10.0.0.2 (10.0.0.2) from 10.0.0.1 tap0: 56(84) bytes of data.
64 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=0.297 ms
64 bytes from 10.0.0.2: icmp_seq=2 ttl=64 time=0.226 ms

--- 10.0.0.2 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1003ms
rtt min/avg/max/mdev = 0.226/0.261/0.297/0.035 ms
```

Figure 2: Terminal execution showing TAP binding, Verilator build, and no packet loss.

3. Layer 2 / Layer 3 Wireshark Packet Analysis

Captured ICMP Echo Request Frame (98 Bytes Total):

- **Layer 2 Header (14B):** Dst MAC 02:00:00:00:00:02, Src MAC 02:00:00:00:00:01, EtherType 0x0800 (IPv4).
- **Layer 3 Header (28B):** IPv4 10.0.0.1 → 10.0.0.2 (20B, Len 84B, ID 0x7F6D), ICMP Type 8 (Echo Request, Code 0, Checksum 0xDB80, ID 0x7602, Seq 1).
- **Payload (56B):** 16-byte kernel timestamp + 40-byte repeating pattern `ca fe 00 03`.

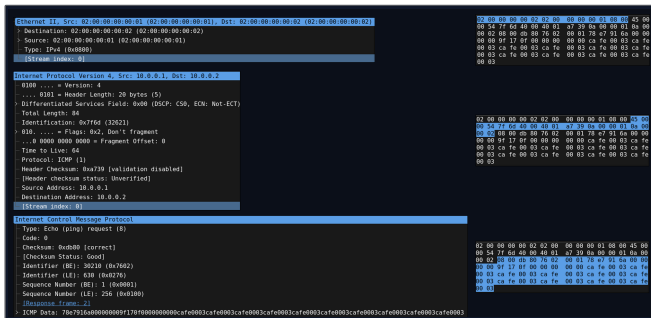


Figure 3: Wireshark packet dissection and correlated hex dumps.

4. Inter-Packet Gap (IPG) & Control Block Encoding (BTF = 0x1E)

Idle Control Block Compression:

- **Idle Input:** MAC outputs TXD = 0x0707070707070707 with TXC = 8'hFF, meaning all 8 lanes have an control signal.
- **Control Sync Header:** PCS encoder sets Sync = 2'b10 (raw_hdr = 2'b10) for Control Blocks.
- **Payload Compression:** Sets BTF = 0x1E (bits [7:0]) and compresses 8 idle bytes into 7-bit PCS codes (raw_payload = 64'h000000000000001E).

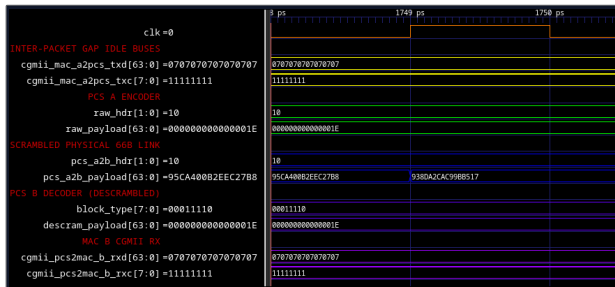


Figure 4: Idle compression: 8 idle bytes encoded into Control Block (BTF = 0x1E, Sync = 2'b10).

5. Frame Start Delimiter (/S/) & Control Block Framing (BTF = 0x78)

Frame Start Delimiter & Block Translation (Cycle 1–2):

- **Start Delimiter Detection:** MAC asserts /S/ = 0xFB on Lane 0 with TXC = 8'h01 (TXD = 0xD5555555555555FB).
- **Control Block Type 0x78:** PCS encoder outputs Sync = 2'b10 and sets BTF = 0x78 on Byte 0.
- **Preamble Passing:** Preamble (0x55) and SFD (0xD5) pass into payload bits [63:8] (raw_payload = 0xD555555555555578).

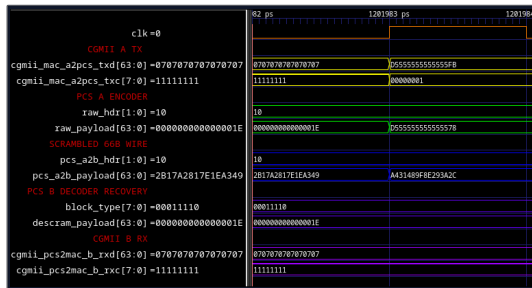


Figure 5: Transition from Idle to Frame Start (/S/ = 0xFB) on Lane 0 mapped to BTF = 0x78.

6. SW-to-HW Correlation: Payload Data Blocks (Sync = 2'b01)

64-Bit Data Streaming & Header Decoding (Cycles 3–12):

- **Data Block Framing:** For data words (TXC = 8'h00), PCS encoder sets Sync = 2'b01 without BTF overhead.
- **Direct Word Mapping:** Unscrambled payload matches Wireshark byte-for-byte (L2 MACs, IPv4 ID 0x7F6D, ICMP Checksum 0xDB80).
- **Throughput:** Transmits 8 bytes/cycle at 100% line rate with 3.125% framing overhead.

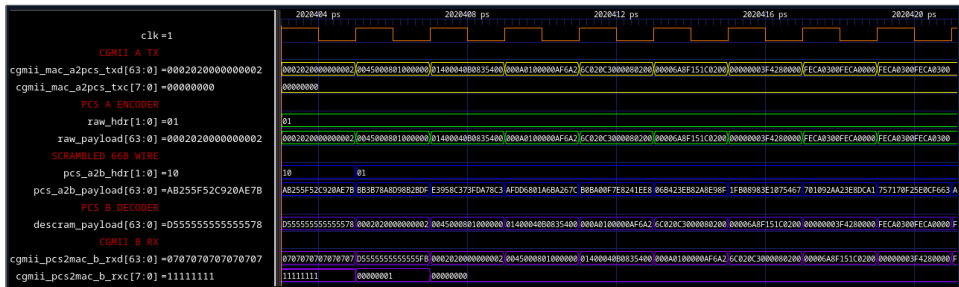


Figure 6: Cycle-by-cycle alignment of L2/L3 packet headers across 64-bit Data Blocks (Sync = 2'b01).

7. 58-Bit LFSR Scrambler & Sync Header Invariance

Self-Synchronizing Scrambler Pipeline ($G(x) = x^{58} + x^{39} + 1$):

- **Bit Recurrence:** $y[i] = x[i] \oplus y[i - 39] \oplus y[i - 58]$, computed in 1 cycle via self-synchronizing LFSR.
- **Sync Header Invariance:** Sync header `Sync[1:0]` bypasses the LFSR to preserve receiver block alignment.
- **Descrambler Recovery:** Descrambler recovers payload without sync tokens: $x'[i] = y[i] \oplus y[i - 39] \oplus y[i - 58] \equiv x[i]$.

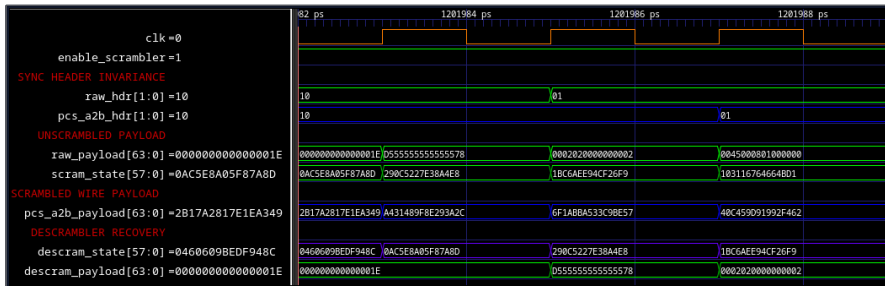


Figure 7: Payload bit randomization via $G(x) = x^{58} + x^{39} + 1$ and exact descrambler recovery.

8. Mathematical Proof of Self-Synchronization

Definitions (IEEE 802.3 Clause 49/82):

- **Transmitter (y):** Encodes data bit $x[i]$ using past wire bits:

$$y[i] = x[i] \oplus y[i - 39] \oplus y[i - 58]$$

- **Receiver (x'):** Decodes incoming wire bit $y[i]$ using the same taps:

$$x'[i] = y[i] \oplus y[i - 39] \oplus y[i - 58]$$

Algebraic Proof:

$$\begin{aligned} x'[i] &= \underbrace{(x[i] \oplus y[i - 39] \oplus y[i - 58])}_{\text{Substitute } y[i]} \oplus y[i - 39] \oplus y[i - 58] \\ &= x[i] \oplus (y[i - 39] \oplus y[i - 39]) \oplus (y[i - 58] \oplus y[i - 58]) \quad (\text{Associative}) \\ &= x[i] \oplus 0 \oplus 0 \quad (\text{Since } A \oplus A = 0) \\ &\equiv x[i] \end{aligned}$$

Takeaway: The receiver recovers clean data with 0 bit errors after 58 bits of wire traffic.

8. Frame Termination (/T/) & BTF Selection (0xE1)

Frame Termination Delimiter & CRC-32 FCS (Cycle 16):

- **CRC-32 FCS:** MAC computes CRC-32 ($P(x) = x^{32} + x^{26} + \dots + 1$) over 98 bytes $\rightarrow$ 0x45210EBC (Lanes 2–5: BC 0E 21 45).
- **Clause 82 BTF Selection:** Delimiter /T/ = 0xFD on Lane 6 (TXC = 8'hC0) maps to BTF = 0xE1 (raw_payload = 0x0045210EBC0300E1).
- **Hardware Reconstruction:** Decoder detects BTF = 0xE1, restores TXC = 8'hC0, and MAC RX validates the FCS.

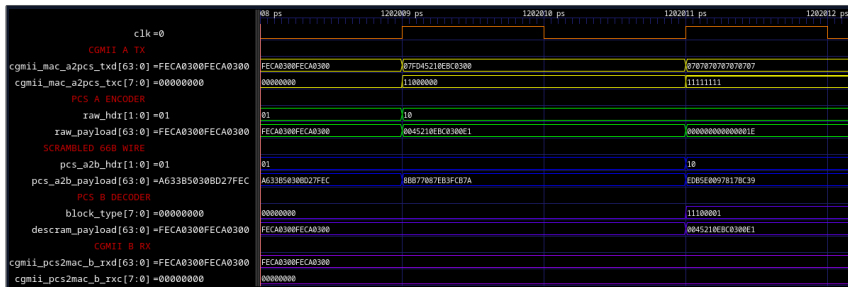


Figure 8: Cycle 16 frame termination: /T/ on Lane 6 mapped to BTF = 0xE1 with CRC-32 FCS (0x45210EBC).

Thank You!